Code Notes on the HNSW Implementation in this Repository

Contents

1 Where the Main Implementation Lives **3**

2 CPU or GPU **3**

3 Public Construction and Query Parameters **3**

4 Distance Spaces and Input Vector Format **4**

5 Internal Node Identifiers and Labels **4**

6 Level Assignment and Entry Point **5**

7 Memory Layout of Node Data **5**

8 How Link Lists Are Stored **6**

9 Graph Construction Path **6**

10 Neighbor Selection and Reciprocal Updates **7**

11 Detailed Control Flow of addPoint **7**

11.1 Clarifications About Nodes, Layers, Edges, Memory Access, and Deletion 8
11.2 Public Wrapper: Replacement Policy 9
11.3 Replacement Path for Deleted Elements 9
11.4 Internal Overload: New Insert or Existing-Label Update 10
11.5 Level Selection and Locking 11
11.6 Writing the Node's Stored Data 11
11.7 First-Node Case 12
11.8 Greedy Descent from Upper Layers 12
11.9 Construction Search and Mutual Connection 13
11.10 Entry Point Update 13
11.11 Clarification: Difference Between addPoint and updatePoint 13
11.12 Short Logical Summary 14

12 Query Path **15**

13 Original Data Handling **15**

14 Index Save File and Load File Format **16**

15 Deletion and Replacement **17**

16 Locking Mechanism During Construction and Deletion **17**
16.1 Why Locks Are Required During HNSW Graph Construction 17
16.2 Meaning of lock_deleted_elements 18
16.3 When a Node Is Marked Deleted 18
16.4 When a Node Is Unmarked Deleted 19
16.5 Relationship Between Deletion, Un-Deletion, and Replacement 19

17 Threading and Locks **20**

18 Mechanism of the C++ Examples **20**
18.1 Basic Search: example_search.cpp 20
18.2 Filtered Search: example_filter.cpp 21
18.3 Multithreaded Search: example_mt_search.cpp 21
18.4 Multithreaded Filtered Search: example_mt_filter.cpp 22
18.5 Epsilon Search: example_epsilon_search.cpp 22
18.6 Multivector Search: example_multivector_search.cpp 23

19 Summary **24**

1 Where the Main Implementation Lives

The HNSW implementation in this checkout is mostly header-only C++. The central class is `hnswlib::HierarchicalNSW<dist_t>` in `hnswlib/hnswalg.h`. The general interfaces, distance-function types, binary read/write helpers, and public algorithm interface are in `hnswlib/hnswlib.h`. The distance spaces are implemented in `hnswlib/space_l2.h` and `hnswlib/space_ip.h`. The Python API is a pybind11 wrapper in `python_bindings/bindings.cpp`.

There is also a separate brute-force baseline in `hnswlib/bruteforce.h`, exposed to Python as `hnswlib.BFIndex`. That file is useful for comparison, but it is not the HNSW graph implementation.

2 CPU or GPU

This implementation is CPU based. The HNSW construction and query code uses ordinary C++ memory allocation, pointer arithmetic, mutexes, priority queues, and CPU distance functions. No CUDA kernels, GPU memory allocations, or GPU launch paths are present in the HNSW code.

The code does contain CPU SIMD optimizations. In `hnswlib.h`, compile-time macros enable `USE_SSE`, `USE_AVX`, and `USE_AVX512` when the compiler and target CPU support them. The L2 and inner-product distance spaces then select vectorized CPU functions, for example `L2SqrSIMD16ExtAVX` or `InnerProductDistanceSIMD16ExtAVX512`. The graph traversal also uses CPU prefetch intrinsics such as `_mm_prefetch` under `USE_SSE`. These are CPU cache/SIMD optimizations, not GPU execution.

3 Public Construction and Query Parameters

The Python binding exposes the HNSW parameters through `Index.init_index`:

```
init_index(max_elements, M=16, ef_construction=200,
          random_seed=100, allow_replace_deleted=False)
```

The same defaults appear in the C++ constructor of `HierarchicalNSW`:

```
HierarchicalNSW(SpaceInterface<dist_t> *s,
               size_t max_elements,
               size_t M = 16,
               size_t ef_construction = 200,
               size_t random_seed = 100,
               bool allow_replace_deleted = false)
```

The important fields are:

- `max_elements_`: fixed capacity of the index allocation. The index can be resized later with `resizeIndex`.
- `cur_element_count`: current number of internal elements. Internal node ids are dense integers from 0 to `cur_element_count - 1`.
- `M_`: user requested graph degree parameter. If `M > 10000` it is capped to 10000.
- `maxM_`: maximum number of neighbors per node on upper layers. It is set to `M_`.
- `maxM0_`: maximum number of neighbors per node on layer 0. It is set to `2 * M_`.

- `ef_construction_`: candidate-list width during insertion. The constructor stores `max(ef_construction , M_)`, so construction search is never narrower than `M_`.
- `ef_`: query-time search width. It defaults to 10 and can be changed from Python with `set_ef` or the writable `ef` property.
- `random_seed`: seeds the level generator and the update probability generator.
- `allow_replace_deleted_`: enables reusing slots that were marked deleted.

The search call exposes:

```
knn_query(data, k=1, num_threads=-1, filter=None)
```

The actual base-layer search width is `max(ef_, k)`. Thus `ef` should be at least `k`, and larger values usually improve recall at the cost of more distance computations.

4 Distance Spaces and Input Vector Format

The public Python constructor is:

```
hnswlib.Index(space, dim)
```

The supported HNSW spaces are:

- `l2`: squared Euclidean distance, implemented by `L2Space` in `space_l2.h`.
- `ip`: inner-product distance `1.0 - dot(a,b)`, implemented by `InnerProductSpace` in `space_ip.h`.
- `cosine`: uses the inner-product space internally, but the Python binding normalizes vectors before insertion and query.

The HNSW class stores a generic distance-function pointer:

```
DISTFUNC<dist_t> fstdistfunc_;
void *dist_func_param_;
size_t data_size_;
```

For the normal Python bindings, `dist_t` is `float` and the stored vectors are contiguous `float32` vectors of length `dim`. The binding converts input arrays with `py::array_t<dist_t, py::array::c_style` py::array::forcecast¿—, checks the dimensionality, optionally normalizes cosine vectors, and passes a row pointer into `HierarchicalNSW::addPoint`.

After insertion, the index owns its own copy of the vector bytes. It does not keep a reference to the original NumPy array or original input buffer.

5 Internal Node Identifiers and Labels

The implementation separates external labels from internal node ids.

- External labels have type `labeltype`, defined as `size_t` in `hnswlib.h`.
- Internal ids have type `tableint`, defined as `unsigned int` in `hnswalg.h`.

- `label_lookup_` maps external labels to internal ids.
- `getExternalLabel(internal_id)` reads the stored external label from the node's memory block.

If the Python user does not provide ids in `add_items`, the binding uses sequential labels. If the user provides ids, those become the external labels. Search returns labels, not internal ids.

6 Level Assignment and Entry Point

Each node receives a random maximum layer from:

```
int getRandomLevel(double reverse_size) {
    double r = -log(U(0,1)) * reverse_size;
    return (int) r;
}
```

The constructor sets:

```
mult_ = 1 / log(1.0 * M_);
revSize_ = 1.0 / mult_;
```

The sampled level is stored in `element_levels_[internal_id]`. The index also keeps:

- `maxlevel_`: highest level currently present in the graph.
- `enterpoint_node_`: internal id used as the starting point for top-down graph traversal.

The first inserted node becomes the entry point. If a later inserted node has a higher level than the previous maximum, it becomes the new entry point and `maxlevel_` is updated.

7 Memory Layout of Node Data

The most important storage decision is that layer 0 links, vector data, and external label are stored together in one large contiguous allocation:

```
char *data_level0_memory_;
```

For each node, the byte layout is:

```
[ level-0 link list ][ raw vector bytes ][ external label ]
```

The constructor computes:

```
size_links_level0_ = maxM0_ * sizeof(tableint)
                   + sizeof(linklistsizeint);
size_data_per_element_ = size_links_level0_
                       + data_size_
                       + sizeof(labeltype);
offsetLevel0_ = 0;
offsetData_ = size_links_level0_;
label_offset_ = size_links_level0_ + data_size_;
```

Therefore:

- `get_linklist0(id)` returns the beginning of the layer-0 neighbor list for node `id`.

- `getDataByInternalId(id)` returns the raw vector bytes for node `id`.
- `getExternalLabeLp(id)` returns the label address for node `id`.

Upper-layer links are stored separately:

```
char **linkLists_;
size_links_per_element_ = maxM_ * sizeof(tableint)
                        + sizeof(linkliststsizeint);
```

For a node with level greater than zero, `linkLists_[id]` points to a block containing that node's link lists for levels 1 through `element_levels_[id]`. The helper `get_linklist(id, level)` indexes into this block with:

```
linkLists_[id] + (level - 1) * size_links_per_element_
```

8 How Link Lists Are Stored

Each link list starts with a small count field, followed by an array of neighbor internal ids:

```
[ link-list count ][ neighbor id 0 ][ neighbor id 1 ] ...
```

The neighbor ids are `tableint` values. The count is manipulated through:

```
getListCount(linklistsizeint *ptr)
setListCount(linklistsizeint *ptr, unsigned short int size)
```

Layer 0 can hold up to `maxM0_ = 2M` neighbors per node. Upper layers can hold up to `maxM_ = M` neighbors per node.

The graph is therefore an adjacency-list graph, but it is stored in compact manual memory blocks rather than in `std::vector<std::vector<int>>`. During traversal, the code obtains a pointer to the link list, reads the count, and then scans the neighbor id array.

9 Graph Construction Path

Insertion starts in `HierarchicalNSW::addPoint(data_point, label, level)`.

The insertion path is:

1. Check whether the label already exists in `label_lookup_`. If it exists, the node is updated instead of creating a duplicate.
2. Assign a new dense internal id from `cur_element_count`.
3. Sample the node level with `getRandomLevel(mult_)` unless a positive level is explicitly supplied.
4. Zero the node's level-0 memory block.
5. Copy the external label into the label region.
6. Copy the input vector bytes into the data region.
7. Allocate upper-level link memory if the sampled level is greater than zero.
8. From the current entry point, greedily descend upper levels until the insertion level is reached.

9. For each level from `min(curlevel, maxlevelcopy)` down to zero, run `searchBaseLayer` with `ef_construction_` width to collect candidate neighbors.

10. Use `mutuallyConnectNewElement` to select neighbors, write the new node's outgoing links, and update reciprocal links in neighboring nodes.

The construction search uses two priority queues. One queue stores candidates to explore, and another stores the best candidates found so far. The visited state is kept in a reusable `VisitedList` from `visited_list_pool_`.

10 Neighbor Selection and Reciprocal Updates

The code does not simply connect to the nearest `M` candidates. It applies the HNSW neighbor diversification heuristic in `getNeighborsByHeuristic2`. Candidates are considered from nearest to farthest. A candidate is kept only if it is not closer to an already selected neighbor than it is to the query/new point. This tends to preserve angular or spatial diversity in the graph.

`mutuallyConnectNewElement` then:

- Writes the selected neighbors into the new node's link list at that level.
- For every selected neighbor, locks that neighbor's link list and tries to add the new node as a reciprocal connection.
- If the neighbor has free capacity, it appends the new node.
- If the neighbor is full, it builds a candidate set containing the old neighbors plus the new node, applies the same heuristic, and rewrites the neighbor list.

This means the graph is maintained as approximately bidirectional, but a full neighbor list can be pruned and rewritten when a new reciprocal edge is better according to the heuristic.

11 Detailed Control Flow of addPoint

The public insertion method is:

```
void addPoint(const void *data_point,
              labeltype label,
              bool replace_deleted = false)
```

This is the method called by the examples through statements such as:

```
alg_hnsw->addPoint(data + i * dim, i);
```

The pointer `data_point` is a generic pointer to the vector bytes that should be inserted. It is declared as `const void *` because the HNSW class does not need to know whether the vector came from a `float*`, `char*`, or another caller-owned byte buffer. The distance space supplies `data_size_` and the distance function, so `addPoint` can copy exactly the required number of bytes into the index.

11.1 Clarifications About Nodes, Layers, Edges, Memory Access, and Deletion

How the HNSW node data is stored. Each inserted multidimensional vector becomes one HNSW node. In the examples, one row of the original data matrix corresponds to one node. For example:

```
alg_hnsw->addPoint(data + i * dim, i);
```

Here `data + i * dim` is the caller's pointer to row `i`. The HNSW node does not keep this original pointer as the permanent node pointer. Instead, `addPoint` copies the vector bytes into HNSW-owned memory:

```
memcpy(getDataByInternalId(cur_c), data_point, data_size_);
```

Therefore, after insertion, the node's vector lives inside `data_level0_memory_`. The original array used by the example is only the input source. The index can still search after the caller's original array is gone, as long as the index itself has not been destroyed.

How one node is managed across multiple layers. One node can appear in multiple HNSW layers, but the vector data is not duplicated once per layer. The node has exactly one internal id `cur_c` and one stored vector. The layer membership is stored in:

```
element_levels_[cur_c] = curlevel;
```

Every node exists at layer 0. If `curlevel = 3`, then that same node id participates in layers 0, 1, 2, and 3. What changes from layer to layer is the node's neighbor list. Layer 0 neighbors are stored inside the node's main record in `data_level0_memory_`. Upper-layer neighbors are stored in `linkLists_[cur_c]`:

```
get_linklist0(cur_c) // layer 0 links
get_linklist(cur_c, level) // links for level >= 1
```

So the implementation does not create separate vector objects for the same node in different layers. It creates separate adjacency lists for the same internal node id at different layers.

How neighbors and connections are stored. Neighbors are stored as adjacency lists of internal node ids, not as linked-list objects and not as explicit edge pairs like `(1, 3)`. Each link list has this memory format:

```
[ neighbor_count ][ neighbor_id_0 ][ neighbor_id_1 ] ...
```

For example, if internal node 1 is connected to nodes 3, 8, and 10 on layer 0, then node 1's layer-0 link list stores a count of 3 followed by the internal ids 3, 8, and 10. There is no separate edge object. The reciprocal edge is represented by also placing 1 into the neighbor list of node 3, 8, or 10 when `mutuallyConnectNewElement` updates the graph.

How neighbor vectors are accessed during insertion and search. Neighbor exploration is pointer-based random access in memory. During construction or search, the graph traversal reads a neighbor id from the current node's adjacency list, then uses that id to compute the address of the neighbor's stored vector:

```
candidate_id = datal[j];
char *candidate_vector = getDataByInternalId(candidate_id);
dist = fstdistfunc_(query_or_new_vector,
                    candidate_vector,
                    dist_func_param_);
```

`getDataByInternalId(candidate_id)` performs address arithmetic:

```
data_level0_memory_
    + candidate_id * size_data_per_element_
    + offsetData_
```

Thus vectors are stored in one large contiguous memory block, but the visited neighbors are usually not visited in contiguous id order. The access pattern is graph-driven random access into `data_level0_memory_`. It does not load vectors from a file during normal insertion or search. Files are only used by `saveIndex` and `loadIndex` for persistence.

Why node deletion is needed. Deletion is needed because real indexes often need to remove or replace items after construction. Physically removing a node from an HNSW graph is expensive because many other nodes may have that internal id in their neighbor lists across multiple layers. This implementation therefore uses lazy deletion:

```
markDelete(label)
```

sets a delete bit in the node's level-0 link-list header. The node remains in the graph so traversal can still pass through it, but search results skip it. If `allow_replace_deleted_` is enabled, the deleted internal id is put into `deleted_elements` and can later be reused by `addPoint(..., replace_deleted = true)`. This saves memory and avoids growing the index capacity when old items are replaced by new items.

11.2 Public Wrapper: Replacement Policy

The first overload has two jobs: enforce the deleted-slot replacement policy and then dispatch to the internal insertion/update routine.

The first check is:

```
if ((allow_replace_deleted_ == false) && (replace_deleted == true)) {
    throw std::runtime_error("Replacement of deleted elements is disabled...");
}
```

This prevents a caller from requesting slot reuse unless the index was constructed with `allow_replace_deleted = true`. In normal insertion mode, `replace_deleted` is false, so the method simply locks the label and forwards to the internal overload:

```
std::unique_lock<std::mutex> lock_label(getLabelOpMutex(label));
if (!replace_deleted) {
    addPoint(data_point, label, -1);
    return;
}
```

`getLabelOpMutex(label)` selects one of the striped label locks. This prevents two threads from concurrently inserting, deleting, or replacing the same external label.

11.3 Replacement Path for Deleted Elements

If `replace_deleted` is true, the method tries to reuse an internal id from the `deleted_elements` set:

```
tableint internal_id_replaced;
std::unique_lock<std::mutex> lock_deleted_elements(deleted_elements_lock);
bool is_vacant_place = !deleted_elements.empty();
if (is_vacant_place) {
    internal_id_replaced = *deleted_elements.begin();
```

```
    deleted_elements.erase(internal_id_replaced);
}
lock_deleted_elements.unlock();
```

Here `internal_id_replaced` is an internal graph node id, not the external user label. The set stores nodes that were marked deleted earlier. The code removes one id from the set while holding `deleted_elements_lock` so that another thread cannot reuse the same slot.

If no deleted slot is available, the method falls back to normal insertion:

```
if (!is_vacant_place) {
    addPoint(data_point, label, -1);
}
```

If a deleted slot is available, the node storage is reused:

```
labeltype label_replaced = getExternalLabel(internal_id_replaced);
setExternalLabel(internal_id_replaced, label);

std::unique_lock<std::mutex> lock_table(label_lookup_lock);
label_lookup_.erase(label_replaced);
label_lookup_[label] = internal_id_replaced;
lock_table.unlock();

unmarkDeletedInternal(internal_id_replaced);
updatePoint(data_point, internal_id_replaced, 1.0);
```

This does not allocate a new node id. Instead, it changes the stored external label, updates the label lookup table, clears the deletion mark, and then calls `updatePoint`. `updatePoint` copies the new vector bytes into the old node's data region and repairs local graph connections.

11.4 Internal Overload: New Insert or Existing-Label Update

The actual graph insertion is handled by:

```
tableint addPoint(const void *data_point,
                  labeltype label,
                  int level)
```

The `level` argument is normally `-1`, meaning "sample a random HNSW level." Tests or special callers can pass a positive level to force a node level.

The method first locks `label_lookup_lock` and checks whether the external label already exists:

```
auto search = label_lookup_.find(label);
if (search != label_lookup_.end()) {
    tableint existingInternalId = search->second;
    ...
    updatePoint(data_point, existingInternalId, 1.0);
    return existingInternalId;
}
```

If the label already exists, no duplicate node is created. The existing node's vector is updated. If replacement of deleted elements is enabled, the method refuses to update an already-deleted node through this path because deleted nodes may be reused by the replacement path.

If the label is new, the method checks capacity:

```
if (cur_element_count >= max_elements_) {
    throw std::runtime_error("The number of elements exceeds the specified limit");
}
```

Then it assigns a dense internal id:

```
cur_c = cur_element_count;
cur_element_count++;
label_lookup_[label] = cur_c;
```

`cur_c` is the internal id used inside graph adjacency lists. `label` is the external id returned to users in search results.

11.5 Level Selection and Locking

After the internal id is reserved, the method locks the new node's link-list mutex:

```
std::unique_lock<std::mutex> lock_el(link_list_locks_[cur_c]);
```

It then chooses the node level:

```
int curlevel = getRandomLevel(mult_);
if (level > 0)
    curlevel = level;
element_levels_[cur_c] = curlevel;
```

The sampled value controls how many upper-layer link lists this node will own. Every node always has level 0. Nodes with `curlevel > 0` also have link lists for levels 1 through `curlevel`.

The method then takes the global lock and snapshots the current maximum graph level:

```
std::unique_lock<std::mutex> templock(global);
int maxlevelcopy = maxlevel_;
if (curlevel <= maxlevelcopy)
    templock.unlock();
```

If the new node is not higher than the current maximum level, the global lock can be released early. If the new node is higher, the lock is retained until the entry point and `maxlevel_` are updated.

11.6 Writing the Node's Stored Data

The method snapshots the current entry point:

```
tableint currObj = enterpoint_node_;
tableint enterpoint_copy = enterpoint_node_;
```

Then it initializes the new node's level-0 storage:

```
memset(data_level0_memory_ + cur_c * size_data_per_element_
      + offsetLevel0_, 0, size_data_per_element_);
```

The pointer expression computes the start address of node `cur_c`'s contiguous record in `data_level0_memory_`. That record contains:

```
[ level-0 links ][ vector bytes ][ external label ]
```

The label and vector are copied into their offsets:

```
memcpy(getExternalLabeLp(cur_c), &label, sizeof(labeltype));
memcpy(getDataByInternalId(cur_c), data_point, data_size_);
```

`getExternalLabeLp(cur_c)` returns a pointer to the label field inside the node record. `getDataByInternalId (cur_c)` returns a pointer to the vector data field. The source `data_point` remains owned by the caller; HNSW stores its own copy.

If the node has upper levels, the method allocates separate upper-layer link storage:

```
if (curlevel) {
    linkLists_[cur_c] =
        (char *) malloc(size_links_per_element_ * curlevel + 1);
    memset(linkLists_[cur_c], 0, size_links_per_element_ * curlevel + 1);
}
```

`linkLists_[cur_c]` is a pointer to a heap block containing link lists for levels 1 through `curlevel`. Level 0 is not stored here; it is stored in `data_level0_memory_`.

11.7 First-Node Case

If the graph is empty, `enterpoint_node_` was initialized to `-1`. The condition:

```
if ((signed)currObj != -1) {
    ...
} else {
    enterpoint_node_ = 0;
    maxlevel_ = curlevel;
}
```

therefore separates normal insertion from first-node initialization. The first node has no neighbors to connect to. It simply becomes the entry point and sets the maximum level.

11.8 Greedy Descent from Upper Layers

For every node after the first one, insertion starts from the current entry point. If the new node's level is lower than the current maximum level, the method greedily descends the upper layers:

```
for (int level = maxlevelcopy; level > curlevel; level--) {
    while (changed) {
        data = get_linklist(currObj, level);
        int size = getListCount(data);
        tableint *datal = (tableint *) (data + 1);
        ...
        if (d < curdist) {
            curdist = d;
            currObj = cand;
            changed = true;
        }
    }
}
```

The goal is to find a good entry point near the new vector before doing the more expensive construction search on lower layers. `get_linklist` returns the adjacency list for `currObj` at that upper level. `data + 1` skips the link-list count header, so `datal` points to the neighbor id array.

11.9 Construction Search and Mutual Connection

After greedy descent, the method connects the new node from `min(curlevel, maxlevelcopy)` down to level 0:

```
for (int level = std::min(curlevel, maxlevelcopy); level >= 0; level--) {
    auto top_candidates = searchBaseLayer(currObj, data_point, level);
    ...
    currObj = mutuallyConnectNewElement(data_point, cur_c,
                                        top_candidates, level, false);
}
```

`searchBaseLayer` performs an `ef_construction_`-wide candidate search around `currObj` at the current layer. The result is a priority queue of candidate neighbor internal ids.

`mutuallyConnectNewElement` then:

- applies the neighbor-selection heuristic,
- writes the selected neighbors into the new node's link list,
- inserts reciprocal links into those neighbor nodes,
- prunes a neighbor's link list if it is already full, and
- returns the next closest selected neighbor to use as the entry point for the next lower layer.

If the original entry point was marked deleted, the code explicitly adds it to the candidate queue so connectivity can still be repaired through it, while normal search results still avoid returning deleted nodes.

11.10 Entry Point Update

At the end of insertion, if the new node's level is higher than the previous maximum level, it becomes the new global entry point:

```
if (curlevel > maxlevelcopy) {
    enterpoint_node_ = cur_c;
    maxlevel_ = curlevel;
}
```

Finally, the method returns `cur_c`, the internal id assigned to the inserted or updated node.

11.11 Clarification: Difference Between addPoint and updatePoint

Main purpose. `addPoint` is the public insertion entry point. It decides whether the call should create a new node, update an existing label, or reuse a deleted node slot. `updatePoint` is not a general new-node insertion method. It assumes an internal node id already exists and replaces the vector stored at that internal id.

When addPoint is called. The examples and Python bindings call `addPoint` when they want to put data into the index:

```
alg_hnsw->addPoint(data + i * dim, i);
```

For a new label, `addPoint` reserves a new dense internal id, samples the node level, copies the vector and external label into HNSW-owned memory, allocates upper-layer link storage if needed, searches for candidate neighbors, and connects the new node into the graph.

When updatePoint is called. `updatePoint` is called from inside `addPoint` in two important cases:

- the caller inserts a label that already exists, so the existing node's vector should be updated instead of creating a duplicate node;
- `replace_deleted = true` reuses an internal id from a deleted node slot, so the old node storage receives a new vector and label.

In both cases, the internal id already exists before `updatePoint` starts.

Memory behavior. `addPoint` may allocate and initialize new graph memory. For a new node, it writes the level-0 record in `data_level0_memory_` and may allocate `linkLists_[cur_c]` for upper layers. `updatePoint` does not reserve a new internal id and does not create a new node record. Its first operation is:

```
memcpy(getDataByInternalId(internalId), dataPoint, data_size_);
```

That line overwrites the vector bytes of an already-existing node.

Graph behavior. `addPoint` connects a new node by searching from the entry point and calling `mutuallyConnectNewElement` layer by layer. `updatePoint` repairs graph neighborhoods around an existing node. It gathers one-hop and two-hop candidates around the updated node, recomputes neighbor lists for affected neighbors using the HNSW heuristic, and then calls `repairConnectionsForUpdate` to reconnect the updated point from the global entry point down to its own level.

Node id behavior. `addPoint` can increase `cur_element_count` when it creates a new node. `updatePoint` never increases `cur_element_count`. It keeps the same internal id and changes the stored vector, and sometimes the surrounding graph links are adjusted to match the new vector location in the metric space.

Short comparison.

Question	**addPoint**	**updatePoint**
Creates a new internal id?	Yes, for a new label.	No. Uses an existing internal id.
Copies vector bytes?	Yes.	Yes, overwrites existing vector bytes.
Allocates upper-layer link storage?	Yes, for a new node with level greater than 0.	No new node storage allocation.
Used for duplicate labels?	Detects the duplicate and dispatches to update.	Performs the actual vector update.
Used for deleted-slot replacement?	Selects and relabels a deleted slot.	Writes new vector data and repairs graph connections.
Changes graph links?	Builds links for a newly inserted node and reciprocal neighbors.	Repairs links around an existing node after its vector changes.

11.12 Short Logical Summary

The full insertion logic can be read as:

1. Validate whether deleted-slot replacement is allowed.
2. Lock by external label.
3. If replacement is requested, either reuse a deleted internal id or fall back to normal insertion.
4. In the internal overload, update an existing label if it already exists.
5. Otherwise reserve a new internal id and label mapping.
6. Sample or force the HNSW level.
7. Copy the external label and vector bytes into HNSW-owned memory.
8. Allocate upper-layer link storage if needed.
9. If this is the first node, make it the entry point.
10. Otherwise greedily descend upper layers, search candidate neighbors on each insertion layer, and mutually connect the new node.
11. If the new node has the highest level, make it the entry point.

12 Query Path

Querying is implemented by `HierarchicalNSW::searchKnn`.

The query path is:

1. Start at `enterpoint_node_`.
2. Compute the distance from the query to the entry point.
3. For levels `maxlevel_` down to 1, greedily move to any neighbor that is closer to the query. This is a single-best path descent on the sparse upper layers.
4. At level 0, call `searchBaseLayerST` with `max(ef_, k)`.
5. Trim the candidate heap down to `k` results.
6. Convert internal ids back to external labels before returning.

The base-layer search scans adjacency lists from the candidate queue. For each visited node it reads the layer-0 neighbor list, computes distances to unvisited neighbors, and updates the candidate queue and result heap. If there are no deletions and no filter function, the implementation uses a `bare_bone_search` template path to avoid deletion/filter checks.

13 Original Data Handling

The original input vectors are copied into the index at insertion time. The key line is the `memcpy` in `addPoint`:

```
memcpy(getDataByInternalId(cur_c), data_point, data_size_);
```

For Python:

- Input data is accepted as a C-style NumPy array and force-cast to `float`.
- For `l2` and `ip`, the row bytes are copied directly into HNSW memory.
- For `cosine`, the binding normalizes each vector first and stores the normalized vector, not the original unnormalized vector.

The method `get_items` retrieves vectors by external label through `getDataByLabel`. For cosine indexes, this returns the normalized stored vectors because those are the bytes kept by the HNSW structure.

14 Index Save File and Load File Format

`saveIndex(location)` writes a binary file containing both metadata and graph/data memory. The order is:

1. `offsetLevel0_`
2. `max_elements_`
3. `cur_element_count`
4. `size_data_per_element_`
5. `label_offset_`
6. `offsetData_`
7. `maxlevel_`
8. `enterpoint_node_`
9. `maxM_`
10. `maxM0_`
11. `M_`
12. `mult_`
13. `ef_construction_`
14. Raw `data_level0_memory_` for all current elements.
15. For each element, the size of its upper-layer link block followed by that block's bytes.

This file stores the vectors, labels, level-0 graph links, upper-layer graph links, entry point, levels, and construction parameters needed to reconstruct the index. On load, `loadIndex` reconstructs `label_lookup_` by reading each stored external label from `data_level0_memory_`. Query-time `ef_` is reset to 10 when loading through the C++ `loadIndex` path; the README notes that users should set `ef` manually after loading a saved index.

15 Deletion and Replacement

Deletion is lazy. `markDelete(label)` maps the label to an internal id and sets a delete bit inside the level-0 link-list header. It does not physically remove the node from the graph. Searches skip marked nodes in results, but traversal may still pass through them.

If `allow_replace_deleted_` is true, deleted internal ids are stored in `deleted_elements`. A later insertion with `replace_deleted=True` may reuse one of those internal ids, replace the stored label and vector, clear the delete bit, and call `updatePoint` to repair local graph connections.

16 Locking Mechanism During Construction and Deletion

This section clarifies why locks are used while constructing the HNSW graph, what `lock_deleted_elements` means, and how deletion/un-deletion interacts with locking.

16.1 Why Locks Are Required During HNSW Graph Construction

HNSW construction modifies shared graph state. When a new node is inserted, `addPoint` does more than copy a vector. It also reserves an internal id, updates the external-label lookup table, writes the node's level-0 memory record, allocates upper-layer link storage, searches candidate neighbors, writes the new node's neighbor lists, and updates reciprocal neighbor lists in already-existing nodes.

If two threads insert nodes at the same time, they may touch overlapping structures:

- `label_lookup_` maps external labels to internal ids.
- `cur_element_count` assigns the next internal node id.
- `data_level0_memory_` stores each node's level-0 links, vector bytes, and label.
- `linkLists_` stores upper-layer link lists.
- `enterpoint_node_` and `maxlevel_` define the global top-level entry point.
- Existing nodes' adjacency lists may be rewritten when reciprocal edges are added.

Without locks, two insertions could assign the same label mapping, update the same neighbor list at the same time, corrupt a link-list count, or change the entry point while another insertion is using it. The implementation therefore uses different locks for different shared resources:

- `getLabelOpMutex(label)` locks operations involving one external label.
- `label_lookup_lock` protects the label-to-internal-id table.
- `link_list_locks_[id]` protects the adjacency lists of a particular internal node.
- `global` protects global entry-point and maximum-level changes.
- `deleted_elements_lock` protects the set of reusable deleted node ids.

The important design idea is that the graph is not locked with one single large lock for every operation. Instead, the code locks smaller pieces of state, such as one label, one node's link list, or the deleted-id set. This allows more concurrency while still protecting memory that can be modified by multiple threads.

16.2 Meaning of lock_deleted_elements

The statement:

```
std::unique_lock<std::mutex> lock_deleted_elements(deleted_elements_lock);
```

creates a scoped lock object named `lock_deleted_elements`. The mutex being locked is `deleted_elements_lock`. The protected data structure is:

```
std::unordered_set<tableint> deleted_elements;
```

`deleted_elements` stores internal node ids that were marked deleted and are available for possible replacement. The lock is needed because one thread may mark a node deleted while another thread is trying to reuse a deleted slot in `addPoint(..., replace_deleted = true)`.

In the replacement path, the code checks and removes a reusable id while holding this lock:

```
bool is_vacant_place = !deleted_elements.empty();
if (is_vacant_place) {
    internal_id_replaced = *deleted_elements.begin();
    deleted_elements.erase(internal_id_replaced);
}
lock_deleted_elements.unlock();
```

This prevents two insertions from taking the same deleted internal id. After one thread erases `internal_id_replaced` from the set, no other thread can pick that same id as a vacant slot.

16.3 When a Node Is Marked Deleted

A node is marked deleted when the user calls:

```
markDelete(label)
```

The public method first locks operations for that external label:

```
std::unique_lock<std::mutex> lock_label(getLabelOpMutex(label));
```

Then it locks `label_lookup_lock` long enough to translate the external label into an internal id:

```
auto search = label_lookup_.find(label);
tableint internalId = search->second;
lock_table.unlock();
```

After that, `markDeletedInternal(internalId)` sets the deletion bit:

```
unsigned char *ll_cur = ((unsigned char *)get_linklist0(internalId)) + 2;
*ll_cur |= DELETE_MARK;
num_deleted_ += 1;
```

The deletion mark is stored inside the level-0 link-list header, not in a separate node object. This is why deletion is described as lazy: the node is not physically removed from the graph, and its neighbor links are not erased. Search can still traverse through the node, but it should not return the node as a result.

If replacement is enabled, the internal id is inserted into `deleted_elements`:

```
if (allow_replace_deleted_) {
    std::unique_lock<std::mutex> lock_deleted_elements(deleted_elements_lock);
    deleted_elements.insert(internalId);
}
```

Here `deleted_elements_lock` protects the reusable-id set while the deleted node id is added.

16.4 When a Node Is Unmarked Deleted

A node is unmarked when the user calls:

```
unmarkDelete(label)
```

The public path is similar to deletion: lock by label, find the internal id in `label_lookup_`, then call `unmarkDeletedInternal`.

The internal method clears the deletion bit:

```
unsigned char *ll_cur = ((unsigned char *)get_linklist0(internalId)) + 2;
*ll_cur &= ~DELETE_MARK;
num_deleted_ -= 1;
```

If replacement is enabled, the node must also be removed from the reusable deleted-id set:

```
if (allow_replace_deleted_) {
    std::unique_lock<std::mutex> lock_deleted_elements(deleted_elements_lock);
    deleted_elements.erase(internalId);
}
```

This lock prevents a race where one thread unmarks the node while another thread is trying to reuse it as a deleted slot.

16.5 Relationship Between Deletion, Un-Deletion, and Replacement

The three operations share the same state:

- the delete bit inside the node's level-0 link-list header,
- `num_deleted_`, which counts deleted nodes, and
- `deleted_elements`, which stores reusable internal ids when replacement is enabled.

Marking deletion sets the bit and optionally inserts the internal id into `deleted_elements`. Unmarking deletion clears the bit and optionally erases the internal id from `deleted_elements`. Replacement insertion removes an id from `deleted_elements`, assigns a new external label to that same internal id, clears the deletion mark, copies new vector bytes into the old node storage, and repairs local connections with `updatePoint`.

The locking relationship is therefore:

- label locks prevent conflicting operations on the same external label,
- `label_lookup_lock` protects label-to-internal-id lookup and updates, and
- `deleted_elements_lock` protects the pool of deleted ids that can be reused.

This is why `lock_deleted_elements` is specifically required: the deleted-id pool is shared by delete, un-delete, and replacement insertion.

17 Threading and Locks

The implementation is CPU multithreaded at the Python batch level. The binding uses `ParallelFor` to distribute rows across threads for `add_items` and `knn_query`. Inside the HNSW class, thread safety is handled with several lock groups:

- `label_op_locks_`: striped locks for operations on external labels.
- `label_lookup_lock`: protects the label-to-internal-id map.
- `link_list_locks_`: one mutex per internal node for link list updates and reads during construction/update.
- `global`: protects global entry-point and maximum-level changes during insertion.
- `deleted_elements_lock`: protects the reusable deleted-id set.

The README states that `add_items` is thread-safe with other `add_items` calls, and `knn_query` is thread-safe with other `knn_query` calls. However, `add_items` and `knn_query` are not advertised as thread-safe with each other.

18 Mechanism of the C++ Examples

This section explains the runnable C++ examples in `examples/cpp`. All of these examples create synthetic random data, build an in-memory HNSW index, and then exercise a particular query or maintenance mode. They are small demonstrations of the API rather than full benchmark programs.

18.1 Basic Search: example_search.cpp

The basic search example demonstrates the normal HNSW build/query/save/load cycle.

1. It sets the dataset shape and construction parameters: `dim = 16`, `max_elements = 10000`, `M = 16`, and `ef_construction = 200`.
2. It creates an L2 distance space: `hnswlib::L2Space space(dim)`.
3. It creates the graph index: `HierarchicalNSW<float>(&space, max_elements, M, ef_construction )`.
4. It allocates one flat array, `float* data = new float[dim * max_elements]`. The pointer `data` owns a contiguous block of all generated vectors. Vector `i` begins at `data + i * dim`.
5. It inserts every vector with `alg_hnsw->addPoint(data + i * dim, i)`. The second argument `i` is the external label.
6. It queries each vector against the index with `searchKnn(data + i * dim, 1)`. Because the query vector is one of the stored vectors, the expected nearest label is `i`.
7. It counts how often the returned top label equals the query label and prints recall.
8. It saves the index with `saveIndex("hnsw.bin")`, deletes the old index object, reloads from the saved file, and repeats the recall check.

Mechanically, `searchKnn` first performs greedy descent through the upper HNSW layers and then performs a wider candidate search on layer 0. The returned object is a `std::priority_queue` of `(distance, label)` pairs, where `result.top()` is used to read the best label for `k = 1`.

18.2 Filtered Search: example_filter.cpp

The filter example demonstrates label-based filtering during search. It defines:

```
class PickDivisibleIds: public hnswlib::BaseFilterFunctor {
    bool operator()(hnswlib::labeltype label_id) {
        return label_id % divisor == 0;
    }
};
```

This class is a callable filter object. For `PickDivisibleIds pickIdsDivisibleByTwo(2)`, only even labels are allowed.

The index construction is the same as the basic search example: random vectors are generated, copied into the HNSW index with `addPoint`, and labels are assigned from 0 to `max_elements - 1`. The difference is the query call:

```
searchKnnCloserFirst(data + i * dim, k, &pickIdsDivisibleByTwo)
```

The third argument is a pointer to the filter functor. During base-layer search, hnswlib can still traverse graph nodes to preserve connectivity, but a candidate is only accepted into the returned result set if `isIdAllowed(label)` returns true. The example checks every returned label and prints an error if any odd label appears.

`searchKnnCloserFirst` is a convenience wrapper around `searchKnn`. It converts the priority-queue result into a vector ordered from closest to farthest, which is easier to inspect in a loop.

18.3 Multithreaded Search: example_mt_search.cpp

The multithreaded search example uses the same HNSW index type, but distributes insertion and querying across CPU threads. The file defines a local `ParallelFor` helper.

The helper works as follows:

- If `numThreads <= 0`, it uses `std::thread::hardware_concurrency()`.
- If `numThreads == 1`, it runs a normal serial loop.
- Otherwise, it creates `numThreads` worker threads.
- The shared atomic counter `current` hands out row ids using `current.fetch_add(1)`.
- Each worker calls the supplied lambda for one row at a time.
- Exceptions are captured in `lastException` and rethrown after all threads join.

Insertion is parallelized with:

```
ParallelFor(0, max_elements, num_threads, [&](size_t row, size_t threadId) {
    alg_hnsw->addPoint((void*)(data + dim * row), row);
});
```

Here `data + dim * row` is a pointer to the first float of the row's vector. The cast to `void*` matches the generic `addPoint` API, which accepts raw vector bytes. The graph object uses internal locks to make concurrent insertions safe with other insertions.

Querying is also parallelized. Each thread calls `searchKnn(data + dim * row, 1)` for a different row and writes the returned label into `neighbors[row]`. After all threads finish, the main thread computes recall by comparing `neighbors[i]` with `i`.

This example should be read as "parallel batch insert, then parallel batch query." It does not mix insertion and query at the same time.

18.4 Multithreaded Filtered Search: example_mt_filter.cpp

The multithreaded filtered example combines the two previous mechanisms:

- It uses the same `ParallelFor` worker-thread helper as `example_mt_search.cpp`.
- It uses the same `PickDivisibleIds` filter class as `example_filter.cpp`.
- It inserts all vectors in parallel.
- It searches all vectors in parallel with a filter pointer.

The query call is:

```
searchKnn(data + dim * row, k, &pickIdsDivisibleByTwo)
```

The output of `searchKnn` is a priority queue, so the example pops `k` labels from the queue and stores them into a flat result vector:

```
neighbors[row * k + i] = label;
```

This memory layout means query row `row` owns the result segment `neighbors[row * k]` through `neighbors[row * k + k - 1]`. After the parallel query phase, the main thread scans all stored labels and reports an error if a returned label is not divisible by two.

Filtering generally makes search slower than the bare query path because the implementation cannot use the simplest no-deletion/no-filter template branch. It must check whether each candidate label is allowed before accepting it into the result heap.

18.5 Epsilon Search: example_epsilon_search.cpp

The epsilon search example demonstrates a custom stop condition rather than a fixed `k`-nearest-neighbor query. It uses:

```
hnswlib::EpsilonSearchStopCondition<dist_t>
    stop_condition(epsilon2, min_num_candidates, max_elements);

alg_hnsw->searchStopConditionClosest(query_data, stop_condition);
```

The example stores data in a raw byte array:

```
size_t data_point_size = space.get_data_size();
char* data = new char[data_point_size * max_elements];
```

The pointer `data` owns all vector records as bytes. For vector `i`, `point_data = data + i * data_point_size` points to the start of that record. Each coordinate is written by computing a byte offset, casting the address to `float*`, and assigning a float value:

```
char* vec_data = point_data + j * sizeof(float);
*(float*)vec_data = value;
```

epsilon2 is the maximum squared L2 distance for final results. min_num_candidates forces the search to inspect at least a minimum number of candidates before stopping just because the next candidate is outside the epsilon radius. max_elements is passed as the maximum number of candidates, so this example allows the stop condition to return as many matching vectors as needed up to the dataset size.

Inside EpsilonSearchStopCondition, search can stop when either:

- the candidate distance is worse than the current lower bound and the maximum candidate count has been reached, or
- the candidate distance is larger than epsilon2 and at least min_num_candidates have already been collected.

After the graph search finishes, filter_results removes candidates whose distance is greater than epsilon2. The example prints how many vectors were found for each random query.

18.6 Multivector Search: example_multivector_search.cpp

The multivector example demonstrates searching for documents where each document may be represented by multiple vectors. The key difference is the space type:

```
hnswlib::MultiVectorL2Space<docidtype> space(dim);
```

This space stores a normal float vector plus an extra document id field at the end of each data record. Therefore:

```
data_point_size = dim * sizeof(float) + sizeof(docidtype)
```

conceptually, although the code obtains this value through space.get_data_size(). A point record looks like:

```
[ float vector coordinates ][ doc_id ]
```

The example writes the vector coordinates manually into a raw char* buffer and then calls:

```
space.set_doc_id(point_data, doc_id);
```

set_doc_id writes the document id at byte offset vector_size_, immediately after the vector coordinates. During index construction, each vector still receives its own HNSW label, and the example keeps a separate label_docid_lookup map so it can count which returned labels belong to which documents.

The query mechanism uses:

```
hnswlib::MultiVectorSearchStopCondition<docidtype, dist_t>
    stop_condition(space, num_docs, ef_collection);

alg_hnsw->searchStopConditionClosest(query_data, stop_condition);
```

num_docs is the target number of distinct documents to return. ef_collection is the number of distinct candidate documents to maintain during search; larger values usually improve recall but slow the search. The stop condition counts documents, not just individual vectors. It uses the space

object to read each candidate vector's stored document id from the point bytes. It can stop once the next candidate cannot improve the result and `ef_collection` distinct documents have already been collected.

After the result vectors are returned, the example maps each returned label back to a document id and prints both the number of distinct documents found and the number of vectors returned.

19 Summary

In this repository, HNSW is implemented as a CPU-resident, pointer-based, header-only C++ graph index. It stores every inserted vector inside the index memory, stores level-0 graph links beside the vector and label in a contiguous per-node record, stores upper-layer links in per-node heap allocations, and uses dense internal ids for graph edges. Public labels are maintained through an unordered map and are what the user sees in query results. Construction is controlled mainly by `M` and `ef_construction`, while query accuracy and speed are controlled mainly by `ef` and `k`.